

1 Introducción a PowerShell

1.1 Qué é PowerShell?

PowerShell é unha interface de liña de comandos (CLI) con funcións de linguaxe de script. Esta interface trata de substituír varias partes obsoletas dos sistemas Windows:

- O vello intérprete de comandos cmd.exe.
- A linguaxe de scripts Windows Scripting Host e o seu host de liña de comandos cscript.exe.
- Certos intérpretes de comandos ad-hoc coma WMIC.

PowerShell pode executar:

- Comandos, que na súa linguaxe particular chámanse *commandlets* (abreviado coma *cmdlet*).
- Scripts, que polo xeral terán a extensión .ps1.
- Funcións.
- Programas standalone, que a diferenza dos cmdlets se executan no seu propio proceso.

Pero ademais, PowerShell pódese embeber noutras aplicacións, de forma que estas expoñen certas funcionalidades a través del. Exemplos deste tipo de aplicacións son Microsoft Exchange e Microsoft SQL Server.

1.2 Versións de PowerShell

A primeira versión de PowerShell, Windows PowerShell 1.0, estaba baseada en .NET Framework 2.0 e ofreceuse como un compoñente opcional de Windows Server 2008. Tamén se fixeron instaladores para Windows XP SP2, Windows Vista e Windows Server 2003 SP1 [1]. Porén, esta xeración de sistemas Windows xa está obsoleta (non recibe actualizacións de seguridade por parte de Microsoft dende hai anos) e a documentación en liña de PowerShell dispoñible para esta versión é practicamente nula.

Windows PowerShell 2.0 saíu como parte de Windows 7 e Windows Server 2008 R2. Tamén se fixeron instaladores para sistemas operativos da xeración anterior, coma Windows XP SP 3, Windows Vista, Windows Server 2003 e Windows Server 2008 como parte do Windows Management Framework Core, que tamén incluía WinRM 2.0 [2]. Entre as melloras que trouxo esta versión tiñamos máis de 100 cmdlets novos, a posibilidade de executar comandos de forma remota, a aparición da contorna de desenvolvemento PowerShell ISE, a modularización dos scripts, funcións avanzadas etc. Aínda que esta versión de PowerShell tamén está oficialmente marcada coma obsoleta, para nós segue a ser importante xa que en moitos casos podemos querer traballar cos sistemas operativos desta xeración en máquinas virtuais. Ademais, a axuda que ven instalada con esta versión aínda está traducida ao castelán, aínda que a axuda en liña xa non está oficialmente dispoñible.

Pola súa parte, Windows PowerShell 3.0 formou parte de Windows 8.0 e Windows Server 2012. Tamén se fixeron instaladores para Windows 7 SP1, Windows Server 2008 SP1 e

Windows Server 2008 R2 SP1 como parte do Windows Management Framework 3.0 [3]. Non obstante pasouse a requirir o .NET Framework 4.0. Entre as novidades desta versión estaban o Windows PowerShell Workflow, a posibilidade de desconectar e reconectar sesións remotas, e a posibilidade de cargar módulos automaticamente entre outras.

Windows PowerShell 4.0 chegou coa xeración de Windows 8.1 e Windows Server 2012 R2, aínda que tamén se fixeron instaladores para Windows 7 SP1, Windows Server 2008 R2 SP1 e Windows Server 2012, como parte do Windows Management Framework 4.0. Nesta versión pasouse a requirir o .NET Framework 4.5 [3].

O seguinte lanzamento, o de Windows PowerShell 5.0, ocorreu coas primeiras versións de Windows 10. Non obstante, foi rapidamente substituído por Windows PowerShell 5.1, tanto nas seguintes actualizacións de Windows 10 coma en Windows Server 2016 e Windows Server 2019. Tamén hai instaladores para versións anteriores do SO coma Windows 7 SP1, Windows 8.1, Windows 2008 R2 SP1, Windows Server 2012 e Windows Server 2012 R2, ofrecidos como parte do Windows Management Framework 5.1 [3]. Con esta versión Microsoft considerou que Windows PowerShell estaba “*finalizado*”, é dicir, que xa tiña tódalas funcionalidades requiridas para administrar un SO Windows e que, polo tanto, non se ían engadir máis características, salvo pequenas melloras e corrección de problemas.

Porén, en 2018 apareceu unha nova versión de PowerShell chamada PowerShell Core 6.0. Malia o parecido do seu nome, PowerShell Core é bastante distinto ao Windows PowerShell. Para comezar, o PowerShell Core non ten coma obxectivo administrar unicamente sistemas Windows, senón que é unha ferramenta Open Source enfocada á nube, e pódese empregar tanto en sistemas Windows coma en diversas distribucións de GNU/Linux e varias versións de MacOS. Ademais, esta ferramenta non está baseada en .NET Framework, senón nunha nova plataforma chamada .NET Core 2.0. PowerShell Core 6.0 tamén ten un executable distinto ao de Windows PowerShell 5.1, e ambos programas poden convivir perfectamente nun mesmo sistema Windows. Coma desvantaxe, temos que salientar que para manter a compatibilidade entre plataformas, o PowerShell Core 6.0 deixou de incluír moitos dos cmdlets que realizaban funcións específicas dos SO de Microsoft. A meirande parte dos módulos de Windows PowerShell tampouco son compatibles con esta plataforma [4]. Unha actualización chamada PowerShell Core 6.1 saíu coa actualización a .NET Core 2.1. Entre as melloras, temos que esta versión é compatible con máis de 1900 cmdlets de Windows 10 e Windows Server 2019, os cmdlets execútanse de forma máis eficiente e hai máis versións dos SO soportadas [5]. Un tempo despois, chegou unha segunda actualización chamada PowerShell Core 6.2. Esta inclúe maiormente resolución de bugs, mais tamén engadiu algunhas características experimentais [6].

Para este curso, xa está dispoñible PowerShell 7, ao que lle volveron quitar “Core” do nome. Primeiro houbo unha versión 7.0 cunha actualización a .NET Core 3.0 [7] e despois a versión 7.1 cunha actualización a .NET Core 3.1 [8]. Entre os obxectivos desta versión está a de mellorar o número de módulos de Windows que se incorporan, intentando que estea practicamente á par con Windows PowerShell 5.1.

Referencias:

- [1] Microsoft. "Windows PowerShell 1.0 localizado el paquete de instalación para Windows Server 2003 y Windows XP". En liña: <https://support.microsoft.com/es-es/help/926140/windows-powershell-1-0-localized-installation-package-for-windows-serv>. Accedido o 14-04-2019.
- [2] Microsoft. "Windows Management Framework (Windows PowerShell 2.0, WinRM 2.0 y BITS 4.0)". En liña: <https://support.microsoft.com/es-es/help/968929/windows-management-framework-windows-powershell-2-0-winrm-2-0-and-bits>. Accedido o 14-04-2019
- [3] Microsoft. "Requisitos del sistema de Windows PowerShell" En liña: <https://docs.microsoft.com/es-es/powershell/scripting/install/windows-powershell-system-requirements>. Accedido o 14-04-2019.
- [4] Microsoft. "Novedades de PowerShell Core 6.0". En liña: <https://docs.microsoft.com/es-es/powershell/scripting/whats-new/what-s-new-in-powershell-core-60>. Accedido o 14-04-2019.
- [5] Microsoft. "Novedades de PowerShell Core 6.1". En liña: <https://docs.microsoft.com/es-es/powershell/scripting/whats-new/what-s-new-in-powershell-core-61>. Accedido o 14-04-2019.
- [6] Microsoft. "Novedades de PowerShell Core 6.2". En liña: <https://docs.microsoft.com/es-es/powershell/scripting/whats-new/what-s-new-in-powershell-core-62>. Accedido o 09-09-2019.
- [7] Microsoft. "Novedades de PowerShell Core 7.0". En liña: <https://docs.microsoft.com/es-es/powershell/scripting/whats-new/what-s-new-in-powershell-70>. Accedido o 4-2-2021.
- [8] Microsoft. "Novedades de PowerShell Core 7.1". En liña: <https://docs.microsoft.com/es-es/powershell/scripting/whats-new/what-s-new-in-powershell-71>. Accedido o 4-2-2021.

1.3 Versións de PowerShell vs versións de Windows

Por desgraza, aínda que instalemos a mesma versión de Windows PowerShell (p. ex. a 5.1) en dúas edicións diferentes de Windows, pode ocorrer que non dispoñamos dos mesmos cmdlets. Isto é debido a que o instalador de Windows PowerShell en realidade só incorpora algúns cmdlets (o seu "core"), mentres que moitos outros forman parte de módulos que veñen preinstalados co sistema operativo.

Por exemplo, en PowerShell hai unha serie de comandos para traballar con direccións IP nun módulo chamado "nettcpip". O cmdlet *Get-NetIPAddress* deste módulo obtén a IP dun equipo. Aínda que podemos empregar este cmdlet en calquera Windows 8.1 ou Windows 2012 R2, non vai funcionar en Windows 7 ou Windows Server 2008 R2, por moito que instalemos a mesma versión de PowerShell ou unha superior. Isto débese a que estas edicións non dispoñen do devandito módulo "nettcpip". A única alternativa segue a ser continuar empregando os comandos clásicos do CMD, como NETSH e similares.

Algo similar ocorre cos cmdlets do módulo "smbshare". o cmdlet *Get-SmbShare* obtén os recursos compartidos por Samba. Malia que este comando está dispoñible a partir de PowerShell 3.0 en Windows 8 e Windows Server 2012, non poderemos executalo en Windows 7 ou Windows Server 2008 R2. Neste caso o factor condicionante é que en versións antigas de Windows, o Windows Management Instrumentation (WMI) todavía non traballaba co espacio

de nomes ROOT/Microsoft/Windows/SMB, que é o que empregan os cmdlets do módulo “smbshare”. Por ende, nos sistemas operativos vellos non queda máis remedio que seguir usando os comandos clásicos NET VIEW, NET USE e NET SHARE.

Ademais téñense reportado casos nos que un cmdlet existe desde unha certa versión de PS, mais viu con algún fallo que non foi arranxado ata varias versións despois.

Por iso, para realizar as tarefas deste curso imos empregar Windows PowerShell 5.1 nunha máquina con Windows 10 (preferiblemente a v.1903, May Update 2019, que era a máis actual cando se creou este curso). Tamén é posible facer moitas das tarefas propostas con outras versións de Windows e de PowerShell, mais isto pode levar a situacións nas que certos cmdlets dean problemas. En calquera caso, é posible descargar as versións máis recentes de Windows 10 en [1], e lembra que se non queres asociar o teu Windows a unha conta de Microsoft en liña hai trucos como facer toda a instalación desconectado de Internet ou fallar a creación da conta varias veces.

Referencias

[1] Microsoft. "Descargar imagen de disco de Windows 10 (archivo ISO)". En liña: <https://www.microsoft.com/es-es/software-download/windows10ISO>. Accedido o 19-10-2019.</p>

1.4 Executando Windows PowerShell

Para executar Windows PowerShell simplemente hai que escribir o comando PowerShell.exe no CMD, ou ben ir ao menú inicio e buscar “PowerShell”. Nas “Ferramentas Administrativas” tamén é posible escoller “Windows PowerShell Modules” (Módulos de Windows PowerShell).

Para executar Windows PowerShell en modo Administrador, podemos pinchar co botón dereito do rato na súa icona do menú Inicio, e elixir “Executar como Administrador” (“Run As Administrator”). Outra alternativa é abrir un PowerShell normal e empregar o seguinte comando:

```
Start-Process powershell -Verb runas
```

En realidade, o executable de PowerShell ten un montón de opcións para a configuración da súa contorna de execución

```
PowerShell[.exe]
  [-Command { - | <script-block> [-args <arg-array>]
    | <string> [<CommandParameters>] } ]
  [-EncodedCommand <Base64EncodedCommand>]
  [-ExecutionPolicy <ExecutionPolicy>]
  [-File <FilePath> [<Args>]]
  [-InputFormat {Text | XML}]
  [-Mta]
  [-NoExit]
  [-NoLogo]
  [-NonInteractive]
  [-NoProfile]
  [-OutputFormat {Text | XML}]
  [-PSConsoleFile <FilePath> | -Version <PowerShell version>]
  [-Sta]
```

```
[-WindowStyle <style>]
```

Iremos vendo estes argumentos a medida que sexa necesario, pero se fora necesaria unha referencia rápida, pódese atopar facilmente no sitio de Microsoft [1].

Se queremos ver a axuda do executable, o comando é o seguinte:

```
PowerShell[.exe] -Help | -? | /?
```

Para aqueles que tamén desexen probar o PowerShell Core, temos que salientar que o executable deste último non é PowerShell.exe, senón pwsh.exe (o cal permite que poidamos ter instalados os dous á vez). Ademais, os parámetros que pode recibir este executable son lixeiramente distintos aos de Windows PowerShell.

Referencias:

[1] Microsoft. “Ayuda de línea de comandos de PowerShell.exe”. En liña: <https://docs.microsoft.com/es-es/powershell/scripting/components/console/powershell.exe-command-line-help>. Accedido o 14-04-2019.

1.5 Introducción aos cmdlets

A maioría dos cmdlets de PowerShell son unha combinación de palabras en inglés que seguen a estrutura Verbo-Nome, onde o verbo indica a acción a realizar e o nome indica o obxecto sobre o que se realiza a devandita acción. Os verbos máis típicos son:

- Add(engadir)
- Clear (limpar ou baleirar)
- Disable (deshabilitar)
- Enable (habilitar)
- Export (exportar)
- Find (buscar)
- Get (obter)
- Import (importar)
- Invoke (invocar)
- New (crear novo, aínda que este realmente non é un verbo)
- Out (botar, sacar por pantalla, por un medio etc.)
- Register (rexistrar)
- Remove (eliminar)
- Set (establecer)
- Unregister (desrexistrar)

O nome pode ser calquera recurso do sistema (Item, Printer, Computer, Process etc.). Por exemplo, temos así comandos coma *Get-Process* para obter os procesos do sistema, *Invoke-Command* para facer unha invocación dun comando en local ou remoto, *Out-Printer* para sacar a saída dun comando por impresora, etc.

Despois do nome do cmdlet adoita ir unha serie de parámetros. O valor de cada parámetro vén precedido polo seu nome, aínda que este se pode omitir nalgúns casos. O nome ten

que ir precedido por un único guión, a diferenza de Bash onde os nomes de parámetro con máis dunha letra veñen precedidos por dous guións, para poder distinguilos dunha acumulación de parámetros con nomes dunha soa letra.

Anteriormente vimos este cmdlet para executar unha consola de PS como administradores *"Start-Process powershell -Verb runas"*. Pois ben, este cmdlet ten un verbo ("Start"), un obxecto sobre o que se realiza a acción ("Process"), e dous parámetros. O primeiro é o nome do proceso a executar. O nome deste parámetro é "-Name" e o seu valor é "PowerShell", mais neste caso permítese omitir o nome do parámetro. O segundo parámetro é un modificador do proceso a executar. O nome do parámetro é "-Verb" e o seu valor é "RunAs".

É preciso salientar que, en xeral, PowerShell non é sensible a maiúsculas e minúsculas. Ademais, para facilitar a transición das persoas acostumadas a usar o CMD ou o Bash, moitos cmdlets teñen alias. Por exemplo, o Cmdlet para listar os arquivos e cartafol fillos dun certo cartafol é *Get-ChildItem*, mais é posible conseguir o mesmo resultado escribindo "dir" ou "ls", xa que son dous alias do devandito cmdlet.

1.6 Obter axuda sobre os cmdlets

O cmdlet para solicitar axuda sobre un cmdlet é *Get-Help*. A continuación do mesmo, escribiremos o nome do cmdlet en cuestión e, se o desexamos, algunha das seguintes opcións:

- -examples para ver exemplos de uso.
- -detailed para obter información detallada (que fai cada parámetro etc.).
- -full para obter a información detallada e os exemplos á vez.
- -online para que se abra un navegador web e amose a axuda en liña.

Por exemplo, imos buscar axuda do propio cmdlet *Get-Help* con:

```
Get-Help Get-Help -full
```

Como resultado deste cmdlet, é posible obter o seguinte erro, que indica que a axuda non está instalada no noso sistema:

REMARKS

Get-Help cannot find the Help files for this cmdlet on this computer. It is displaying only partial help.

-- To download and install Help files for the module that includes this cmdlet, use Update-Help.

-- To view the Help topic for this cmdlet online, type: "Get-Help Get-Help -Online" or go to <http://go.microsoft.com/fwlink/?LinkID=113316>.

Neste caso, podemos tentar descargar a axuda mediante o cmdlet *Update-Help*, mais para PowerShell 3.0 e posteriores é posible que o cmdlet non faga nada porque a axuda dispoñible na rede non está localizada no noso idioma. Isto se debe a que o número de cmdlets aumentou sensiblemente e Microsoft non está disposto a facer tantas traducións. No seu momento anunciouse que é posible que se fagan algunhas traducións pouco a pouco, máis

primeiro irán os idiomas máis demandados (chinés, xaponés, francés, alemán etc.) [1]. Por tanto, primeiro podemos probar se funciona o comando para descargar a axuda en español:

```
Update-Help -UICulture es-ES
```

E se non funciona, podemos probar a descargar a versión en inglés (EEUU):

```
Update-Help -UICulture en-US
```

Se xorde algún problema tamén podemos intentar empregar este comando cun PowerShell de administrador e coa opción -Force.

```
Update-Help -UICulture en-US -Force
```

Referencias:

[1] Dr. Tobias Weltner. “Localized Help for PowerShell 3.0” En liña: <http://www.powertheshell.com/localized-help-for-powershell-3-0/>. Accedido o 14-04-2019.

1.7 Cmdlets de información básica

Malia ser capaces de ver a axuda dun cmdlet, aínda non sabemos que cmdlets hai dispoñibles no sistema. Para ver un listado dos cmdlets dispoñibles, podemos executar o seguinte cmdlet:

```
Get-Command
```

O parámetro -Name serve para sacar información dun certo cmdlet. Tamén podemos filtrar os comandos por tipo para saber cales son os verdadeiros cmdlets (escritos en .NET, con comprobación de parámetros, etc.) e cales funcións avanzadas (escritas en PowerShell).

```
Get-Command -Name Get-Help
```

```
Get-Command -CommandType Cmdlet
```

```
Get-Command -CommandType Function
```

Outro cmdlet básico de interese é o que permite ver a versión do PowerShell entre outras informacións do Console Host:

```
Get-Host
```

Aínda que este cmdlet nos saca o idioma do sistema e da interface gráfica, tamén temos cmdlets básicos para isto.

```
Get-Culture
```

```
Get-UiCulture
```

1.8 Tubos (pipes), redireccións e comandos aniñados

Ao igual que en bash, PowerShell tamén soporta que a saída dun cmdlet se envíe como entrada doutro cmdlet a través dun tubo (*pipe*), simbolizado polo carácter de barra vertical “|”. O cmdlet que recibe a saída do anterior debe aceptala expresamente como entrada dun dos seus parámetros. Por iso, cando miramos a documentación detallada dun cmdlet, no

apartado relativo a cada parámetro hai un campo chamado “*Accept pipeline input*” que indica se se permite darlle un valor a través do pipeline.

Ademais, PowerShell conta cunha serie de cmdlets que serven especificamente para operar coa saída doutros comandos, concretamente:

- Formatar a saída (p. ex.: *Format-List*, *Format-Table*...)
- Seleccionar campos da mesma (p. ex.: *Select-Object*)
- Convertela a outros formatos (p. ex.: *ConvertTo-Csv*, *ConvertTo-Html*, *ConvertTo-Json*, *ConvertTo-Xml*...)
- Exportala a outros formatos (p. ex.: *Export-Csv*)
- Redirixila (*Out-Default*, *Out-File*, *Out-Host*, *Out-Null*, *Out-Printer*, *Out-String*, *Out-GridView*...) etc.

Por exemplo:

```
Get-Command | Select-Object name
```

```
Get-Command | Format-List
```

```
Get-Command | Fromat-Table
```

```
Get-Command | Select-Object commandtype,name,version,source |  
ConvertTo-Csv
```

```
Get-Command | Select-Object commandtype,name,version,source |  
Export-Csv -Path out.csv
```

```
Get-Command | Out-GridView
```

```
Get-Command | Out-Printer
```

```
Get-Command | Out-File out.txt
```

Tamén é posible usar as redireccións de saída típicas de bash, (“>” e “>>”), pero estas non son máis que aliases de *Out-File* e de *Out-File* coa opción *-Append* respectivamente.

```
Get-Command > out.txt
```

```
Get-Command >> out.txt
```

Por último, é posible usar o resultado dun comando como valor do parámetro doutro comando. Para isto, empréganse os parénteses. Por exemplo, supoñamos que temos 10 cartafolios con nomes do 0 ao 9 e queremos listar un dos cartafolios, escollido aleatoriamente. O cmdlet *Get-ChildItem* serve para listar un cartafol. Pola súa parte, o cmdlet *Get-Random* xera un número aleatorio. Poderíamos combinar os cmdlets da seguinte forma:

```
Get-ChildItem (Get-Random -Maximum 10)
```

Neste caso, como *Get-ChildItem* acepta a entrada por pipeline, sería o mesmo que facer:

```
Get-Random -Maximum 10 | Get-ChildItem
```


Non todos os comandos aceptan entrada por pipeline, e aínda que a acepten, esta pode ir como moito a un parámetro. Por iso aniñar comandos mediante parénteses pode ser unha alternativa moi útil, xa que en cada parámetro podemos poñer un comando entre parénteses para empregar o seu resultado.

1.9 Parámetros comúns dos cmdlets

Todos os cmdlets de PowerShell dispoñen dunha serie de parámetros comúns:

- -Debug: o cmdlet devolve información de depuración
- -ErrorAction: elixe que facer en caso de erro non crítico (continuar, ignorar, preguntar, parar, etc.).
- -ErrorVariable: elixe en que variable se vai gardar a información de erro en caso de que se produza un.
- -InformationAction: Establece que facer cando haxa que mostrar información (p. ex.: mediante o comando Write-Information). Se non se especifica, tómase un valor por defecto establecido nunha variable chamada \$InformationPreference. Este parámetro está dispoñible desde PowerShell 5.
- -InformationVariable: Garda nunha variable a cadea resultante de executar Write-Information (Dispoñible desde PowerShell 5).
- -OutVariable: garda os obxectos que devolve o comando na variable especificada por este parámetro.
- -OutBuffer: establece o número de obxectos que se acumulan nun buffer antes de mandalos ao pipeline (por defecto: 0).
- -PipelineVariable: Almacena o elemento actual do pipeline na variable indicada (dispoñible desde PowerShell 3).
- -Verbose: o comando devolve información extra.
- -WarningAction: elixe que facer en caso de advertencia (continuar, preguntar, parar...).
- -WarningVariable: elixe en que variable se vai gardar a información de advertencia en caso de que se produza unha.

Iremos vendo os parámetros máis importantes a medida que los vaíamos necesitado (maiormente na sección de xestión de erros nos scripts).

Ademais destes parámetros, PowerShell inclúe outros dous que nos axudan a protexernos contra erros: o parámetro -Confirm e o parámetro -WhatIf. Estes parámetros non son comúns a todos os cmdlets, mais si que son aceptados por moitos deles.

O parámetro -Confirm simplemente fai que un comando pida confirmación ao usuario antes de ser executado. Por exemplo, supoñamos que queremos copiar o arquivo C:\a.txt no mesmo directorio e chamarlle b.txt. O comando de PowerShell normal sería:

```
Copy-Item -Path "C:\a.txt" -Destination "C:\b.txt"
```

Mais se activamos a petición de confirmación, sería:

```
Copy-Item -Path "C:\a.txt" -Destination "C:\b.txt" -Confirm
```

O parámetro `-WhatIf`, pola súa parte, fai unha simulación do que pasaría se se executa o comando, aínda que non o executa realmente. Isto serve para ver que obxectos se verían afectados. Por exemplo, se usamos o comando `New-Item` para crear un ficheiro `C:\proba.txt`, o comando de PowerShell normal sería:

```
New-Item -Path "C:\" -Name "proba.txt" -ItemType File
```

E o ficheiro crearíase a menos que houbese algún problema. Mais se activamos o parámetro `-WhatIf`, o comando sería:

```
New-Item -Path "C:\" -Name "proba.txt" -ItemType File -WhatIf
```

E a saída diría algo coma o seguinte, mais o ficheiro non se crearía:

```
"What if: Performing the operation 'Create File' on target 'Destination: C:\proba.txt'."
```

Pódese atopar unha información máis extensa sobre os parámetros comúns na web de Microsoft [1].

Outro parámetro que temos que salientar, e que non está documentado xunto cos anteriores, é o parámetro `-PassThru`. O que fai este parámetro é forzar aos cmdlets que xeralmente non devolven ningunha saída a devolver unha, de forma que podemos usala para calquera tarefa [2]. Se voltamos ao exemplo da copia de ficheiros, podemos executar:

```
Copy-Item -Path "C:\a.txt" -Destination "C:\b.txt" -PassThru
```

E veremos unha saída coma a seguinte:

Directory: C:\

Mode	LastWriteTime	Length	Name
------	---------------	--------	------

----	-----	-----	
------	-------	-------	--

-a----	15/04/2019	1:21	0 b.txt
--------	------------	------	---------

Referencias:

[1] Microsoft. "About CommonParameters". En liña: https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_commonparameters. Accedido o 15-04-2019

[2] Microsoft "Use the PowerShell PassThru parameter and get back objects". En liña: <https://devblogs.microsoft.com/scripting/use-the-powershell-passthru-parameter-and-get-back-objects/>. Accedido o 15-04-2019.

1.10 Módulos de Windows PowerShell

Como xa se dixo anteriormente, os comandos de PowerShell organízanse en módulos, mais os módulos dispoñibles poden depender da versión de Windows. Por exemplo, non hai módulo `ServerManager` en Windows 7, pero si en Windows Server 2008 R2. Tamén depende dos roles instalados en Windows Server. Por exemplo, para usar o módulo de PowerShell para

configurar o ActiveDirectory en Windows Server 2008 R2, necesítase instalar o rol Active Directory Domain Services (AD DS) ou o rol Active Directory Lightweight Domain Services (AD LDS).

Ademais, algúns módulos de Windows PowerShell están instalados pero non están cargados por defecto. En PS 2.0 tiñamos que importalos explicitamente para poder usar os seus comandos, e estes deixaban de estar dispoñibles cando cerremos a sesión de PowerShell. A partir de PS 3.0, os módulos impórtanse automaticamente ao usar un comando, aínda que esta configuración se pode deshabilitar.

Para listar tódolos módulos importados en PowerShell, podemos empregar o comando:

```
Get-Module
```

Para obter información sobre un módulo en concreto:

```
Get-Module <NOME_DO_MÓDULO>
```

Por exemplo:

```
Get-Module PSScheduledJob
```

```
Get-Module Servermanager
```

Para incluír no listado aqueles que están dispoñibles para seren cargados, usamos a opción -ListAvailable:

```
Get-Module -ListAvailable
```

Para cargar explicitamente un módulo na sesión actual, podemos usar o cmdlet:

```
Import-Module <NOME_DO_MÓDULO>
```

Por exemplo:

```
Import-Module Microsoft.PowerShell.Host
```

E para descárgalo, empregamos o cmdlet:

```
Remove-Module <NOME_DO_MÓDULO>
```

Por exemplo:

```
Remove-Module Microsoft.PowerShell.Host
```

É importante non confundir os cmdlets *Import-Module* e *Remove-Module* cos cmdlets *Install-Module* e *Uninstall-Module* que instalan e desinstalan módulos en Windows.

Tamén podemos querer ver os cmdlets que forman parte de un módulo. Para iso, empregamos o cmdlet:

```
Get-Command -Module <NOME_DO_MÓDULO>
```

Podemos probar un exemplo cun dos módulos cargados por defecto:

```
Get-Command -Module Microsoft.PowerShell.Security
```

1.11 Políticas de execución de scripts

Por defecto, PowerShell non permite a execución de scripts sen asinar dixitalmente por motivos de seguridade, xa que necesitamos ter confianza en quen os escribiu. Mais esta política pódese cambiar. Pero primeiro, é necesario saber cal é a política actual:

```
Get-ExecutionPolicy
```

Se queremos executar os nosos propios scripts, cambiaremos a política mediante o comando *Set-ExecutionPolicy*. Por exemplo, para aceptar os scripts locais pero pedir que se asinen os scripts remotos, empregaremos:

```
Set-ExecutionPolicy remotesigned
```

Por outra banda, para aceptar calquera tipo de script, empregaremos:

```
Set-ExecutionPolicy unrestricted
```

E responderemos “s” (ou “y” na versión en inglés).

Volveremos a insistir sobre este punto nos módulos seguintes, mais agora necesitamos configurar unha política sen restricións coma a do cmdlet anterior para executar os comandos que se propoñen nos seguintes apartados.

1.12 Execución remota de cmdlets e scripts

Nota: para poder probar os cmdlets que se citan neste apartado é preciso ter outra máquina Windows conectada na mesma rede local ou dominio.

PowerShell ten dous mecanismos para executar comandos e scripts de forma remota:

- Executar un comando ou script remotamente de forma illada.
- Abrir unha sesión remota do shell e executar os comandos nesa sesión.

En ambos casos hai uns requisitos previos para que todo funcione correctamente [1]:

- Que o servizo WinRM 2.0 [2] estea correndo na máquina remota. Esta versión do servizo ven incluído en Windows 7 ou superior, así como Windows Server 2008 R2 ou superior.
- Que os portos 5985 (para http), e 5986 (para https) estean abertos no firewall [3].
- Que se permita explicitamente a execución remota de comandos e scripts.
- Que a máquina onde se van a executar os comandos confíe na máquina que llos envía.

Para solucionar os tres primeiros puntos, executamos o seguinte comando:

```
Enable-PSRemoting -force
```

Aínda que en teoría o comando anterior habilita o servizo WinRM, se quixéramos activalo mediante o CMD poderíamos facelo co comando:

WinRM quickconfig

Con isto xa deberían estar activados os servizos necesarios e os correspondentes portos do firewall deberían estar abertos.

En canto á confianza, se as dúas máquinas están no mesmo dominio deberían confiar entre elas de forma automática. En caso de estar nun grupo de traballo ou noutra estrutura, hai que cambiar a configuración do WinRM para habilitar a confianza:

```
Set-Item wsman:\localhost\client\trustedhosts <MÁQUINAS>
```

Se por exemplo, quixeramos confiar en calquera máquina (o cal é perigoso, pero pode simplificar a nosa práctica), podemos usar o comodín *:

```
Set-Item wsman:\localhost\client\trustedhosts *
```

En cambio, se quixeramos confiar só nun par de máquinas chamadas server01 e server02:

```
Set-item wsman:\localhost\client\trustedhosts -value  
server01,server02
```

Unha vez cambiada a confianza, hai que reiniciar o servizo WinRM:

```
Restart-Service WinRM
```

Unha vez reiniciado, podemos comprobar que funcionou correctamente mediante:

```
Test-WSMan <MÁQUINA>
```

Se todo vai ben, xa podemos executar comandos remotamente noutra máquina mediante o cmdlet *Invoke-Command*:

```
Invoke-Command -ComputerName <MÁQUINA> [-Credential <USUARIO>] -  
ScriptBlock {<COMANDOS>}
```

Por exemplo, imos invocar o comando *Get-Culture* na máquina “server01”, co usuario “user01” que pertence ao dominio “domain01”.

```
Invoke-Command -ComputerName server01 -Credential domain01\  
user01 -ScriptBlock {Get-Culture}
```

No caso de querer executar un script, as opcións a empregar son distintas:

```
Invoke-Command -FilePath <FICHEIRO> -ComputerName <MÁQUINA>
```

Por exemplo:

```
Invoke-Command -FilePath c:\test.ps1 -ComputerName Server01
```

O cmdlet *Invoke-Command* serve para executar moitos outros cmdlets remotamente. Porén, hai certos comandos que xa teñen un parámetro *-ComputerName* incorporado que permiten indicar a máquina na que deben executarse: *Restart-Computer*, *Test-Connection*, *Get-EventLog*..., e, polo tanto, non é preciso usar *Invoke-Command* con eles.

Imos ver agora un exemplo práctico do uso de comandos remotos. En apartados anteriores vimos como descargar a axuda dos comandos de PowerShell. Neste apartado imos crear un repositorio para que outras máquinas da mesma rede podan descargar os arquivos de axuda desta máquina. Para iso, supoñamos que o noso equipo se chama "Server01", que está nun dominio chamado "domain01" e que hai un usuario creado neste dominio que se chama "user01". Supoñamos tamén que imos a crear o repositorio nunha carpeta que previamente compartimos na nosa rede (p. ex.: \\Server01\Share\PSHelp).

Primeiro, na máquina descargamos a axuda mediante o cmdlet *Update-Help* tal e como vimos anteriormente. Despois, procedemos a gardala na carpeta compartida mediante o cmdlet:

```
Save-Help -DestinationPath \\Server01\Share\PSHelp -Credential "domain01\user01"
```

Agora, supoñendo que temos un arquivo chamado computers.txt no que temos a lista dos equipos aos que queremos transferir a axuda, facemos:

```
Invoke-Command -ComputerName (Get-Content computers.txt) -ScriptBlock {Update-Help -SourcePath \\Server01\Share\PSHelp -Credential "domain01\user01"}
```

Para un estudo mais pormenorizado sobre a execución remota de scripts, é posible visitar a web de Microsoft [4].

Referencias:

[1] Microsoft "About remote requirements". En liña: <https://technet.microsoft.com/en-us/library/dd315349.aspx>. Accedido o 15-04-2019.

[2] Microsoft. "Windows Remote Management". En liña: <https://docs.microsoft.com/en-us/windows/win32/winrm/portal>. Accedido o 21-20-2019.

[3] Chris We, Microsoft. "What Port does PowerShell remoting use?". En liña: <https://blogs.technet.microsoft.com/christwe/2012/06/20/what-port-does-powershell-remoting-use/>. Accedido o 15-04-2019.

[4] Microsoft. "Ejecutar Comandos Remotos". En liña: <https://docs.microsoft.com/es-es/powershell/scripting/core-powershell/running-remote-commands>. Accedido o 15-04-2019.

1.13 Sesións remotas

Se queremos executar moitos comandos de forma remota, e o resultado duns vai ser empregado noutros, quizais sexa preferible abrir unha sesión remota. Para isto, executaremos o cmdlet:

```
Enter-PSSession <MÁQUINA>
```

Por exemplo:

```
Enter-PSSession "Server01"
```

Esta sesión non é persistente. Para crear unha sesión persistente, é preciso empregar o cmdlet:

```
New-PSSession -ComputerName <MÁQUINAS>
```

E tamén é conveniente gardar a devandita sesión nunha variable (p. ex.: \$s) para poder seguila usando máis adiante. Por exemplo:

```
$s = New-PSSession -ComputerName Server01, Server02
```

E unha vez creada a sesión, podemos usar o comando *Invoke-Command* con ela:

```
Invoke-Command -Session $s {<COMANDOS>}
```

Cando desexemos finalizar a sesión remota, empregaremos o cmdlet:

```
Exit-PSSession <MÁQUINA>
```

Por exemplo:

```
Exit-PSSession Server01
```

2 Cmdlets máis comúns de Windows PowerShell

No capítulo anterior vimos os fundamentos básicos de Windows PowerShell, e tamén comezamos a ver uns cantos comandos. Neste capítulo, estudaremos algúns dos comandos máis comúns divididos pola súa temática. Non se ofrecerá un manual extensivo de cada comando e cada parámetro, xa que para iso está a documentación oficial, senón que simplemente veremos os parámetros máis comúns e algúns exemplos de uso frecuente. Con propósito de ilustrar os parámetros, incluiremos en cada cmdlet o seu nome xunto co seu valor, aínda que en moitos casos o nome é opcional. Finalmente, ofreceremos un compendio de comandos de PowerShell para que a procura de información sobre os mesmos sexa rápida.

2.1 Cmdlets para tratar con ficheiros e cartafóles

Os comandos de PowerShell para tratar con ficheiros e cartafóles funcionan de forma bastante similar aos de bash ou cmd, xa que ao final as operacións que soporta o SO son fundamentalmente as mesmas. Mais unha característica de PowerShell é que, a medida que aumentan as versións, trata de unificar distintos tipos de recursos. Por exemplo, algúns destes comandos tamén serven para tratar con enlaces simbólicos, claves de rexistro, certificados, variables, etc. A todos estes recursos PowerShell chámalles ítems, e aos manexadores que tratan cada tipo de recurso, chámalles provedores (providers) [1]. Porén, nós imos centrarnos no tocante aos ficheiros e directorios.

Outra característica destes comandos é que moitos deles comparten varios parámetros, que procedemos a describir a continuación, para non ter que repetir a explicación en cada comando:

- O parámetro `-Path` simboliza a ruta ao recurso. Se a ruta inclúe espazos ou caracteres raros, é mellor envolvela en comiñas dobres, malia que todos estes caracteres tamén se poden escapar un a un mediante o acento invertido `"`"`. Ademais, en moitas

ocasi3ns esta ruta en realidade pode ser unha lista, cos seus elementos separados por comas. Desta forma pode operarse con varios elementos 3 vez. Este par3metro moitas veces soporta comod3ns coma "*" ou "?".

- O par3metro `-LiteralPath` funciona m3is ou menos coma `-Path`, mais non interpreta os caracteres comod3n, o que pode ser moi conveniente en certas ocasi3ns. Un problema deste par3metro 3 que foi introducido en PS con posterioridade, e a maior3a dos comandos que aceptan entrada por pipeline van facelo a trav3s do par3metro `-Path` [2].
- Por outra banda, os par3metros `-Filter`, `-Include` e `-Exclude` serven para elixir qu3 arquivos ou cartafoles deben procesarse. Aqu3 tam3n 3 moi frecuente o uso de comod3ns "*" ou "?". No caso de elementos do sistema de arquivos, o par3metro `-Filter` fai exactamente o mesmo que o par3metro `-Include`, s3 que `-Filter` 3 m3is r3pido. Para outros provedores, `-Filter` pode facer cousas distintas a `-Include` [3].
- Pola s3a parte, o par3metro `-Force` serve para que os comandos non se dete3an se encontran arquivos con atributos especiais tales como ser ocultos ou de s3 lectura.
- Moitos destes comandos te3en documentado o par3metro `-Credential` para facer as operaci3ns en nome doutro usuario, mais na documentaci3n explican que non funciona cos provedores por defecto de Windows PowerShell, e que polo tanto ser3a necesario o soporte dun terceiro.
- Por 3ltimo os comandos de proba `-Confirm` e `-WhatIf` que estudamos en apartados anteriores tam3n adoitan estar dispo3nibles. O par3metro `-PassThru` tam3n est3 dispo3nible nalg3ns.

Comecemos agora a usar os cmdlets. O primeiro 3 saber moverse pola estrutura de directorios. Para iso temos catro comandos. *Get-Location*, *Set-Location*, *Push-Location* e *Pop-Location*. Cabe dicir que estes comandos tam3n serven para navegar por outras estruturas coma as unidades de rede, o rexistro de Windows etc.

```
Get-Location [-PSDrive <DISPOSITIVO>] [-StackName <NOME_PILA>]

Set-Location [-Path <RUTA>] [-StackName <NOME_PILA>]

Push-Location [-Path <RUTA>] [-StackName <NOME_PILA>]

Pop-Location [-StackName <NOME_PILA>]
```

O cmdlet *Get-Location* simplemente nos da a nosa localizaci3n na estrutura actual, tal e como far3a o comando *pwd* de bash, mentres que o comando *Set-Location* accede a unha ruta, coma o comando *cd* de bash e de cmd. Hai que ter en conta que para navegar 3 ra3z dunha unidade 3 necesario incluír o car3cter "\" tralo nome da unidade. Sen3n, non navegar3:

```
Set-Location -Path "C:"

Get-Location

Set-Location -Path "C:\"

Get-Location
```


En xeral, basta escribir o nome do cmdlet *Get-Location* sen parámetros. Se por algunha razón estamos navegando por outra estrutura (p. ex.: o rexistro) e queremos ver a nosa localización na árbore de directorios da unidade C:, teremos que indicar o dispositivo en cuestión mediante o parámetro *-PSDrive*, sen incluír os dous puntos “:.” no nome da unidade.

```
Set-Location HKLM:\Software\Microsoft
```

```
Get-Location
```

```
Get-Location -PSDrive C
```

Outra novidade de PowerShell son os comandos *Push-Location* e *Pop-Location* que nos permiten facer unha navegación en pila, coma os navegadores web. O comando *Push-Location* non só accede a un lugar da mesma maneira que *Set-Location*, senón que tamén almacena este lugar na cima da pila. Pola súa parte, o comando *Pop-Location* non vai ao directorio pai, senón que elimina o lugar que haxa na cima da pila e accede ao lugar que queda na nova cima. Por exemplo:

```
Set-Location C:\
```

```
Push-Location C:\Users\Administrador
```

```
Pop-Location
```

```
Get-Location
```

E en lugar de ir a C:\Users estamos de volta en C:, que era o directorio no que estivemos antes de ir a C:\Users\Administrador.

Ademais PowerShell permite crear varias pilas e traballar con elas simultaneamente a través do parámetro *-StackName*. Por defecto, trabállase cunha única pila sen nome, pero se incluimos este parámetro, automaticamente creárase unha pila co nome proporcionado e xa poderemos empregala cando queiramos.

Á hora de crear un ficheiro ou directorio, PowerShell non distingue con dous comandos distintos como fan bash ou cmd, senón mediante diferentes parámetros dun mesmo comando, chamado *New-Item*:

```
New-Item [-Path <RUTA>] [-Name <NOME>] [-Type <TIPO>] [-Value <CONTIDO>]
```

Por exemplo:

Crear un arquivo de texto exemplo.txt co contido “que tal?” no cartafol actual.

```
New-Item -Path "." -Name "exemplo.txt" -ItemType "file" -Value "que tal?"
```

Crear dous cartafoles, “cartafol1” e “cartafol2” no cartafol actual.

```
New-Item -Path cartafol1, cartafol2 -ItemType "directory"
```

Hai varios aspectos a ver sobre este comando:

- A ruta pode incluír xa o nome do arquivo, co que o parámetro `-Name` sería innecesario.
- O valor do parámetro `-Type` adoita ser `"file"`, para crear arquivos regulares, ou `"directory"`, para crear cartafolios, pero como xa comentamos hai varios valores posibles.
- No parámetro `-Value` podemos engadir o contido do arquivo.

Para comprobar que o contido dos arquivos é o correcto, podemos empregar o comando `Get-Content`, que funcionaría coma o comando `cat` de bash.

```
Get-Content [-Path] <RUTA> [-Delimiter <DELIMITADOR>] [-Include <PATRÓN>] [-Exclude <PATRÓN>] [-TotalCount <NÚMERO>] [-Tail <NÚMERO>]
```

Este cmdlet retorna o contido do ficheiro en forma de colección de obxectos. Polo xeral, cada un destes obxectos representa unha liña do ficheiro, mais é posible indicar outro delimitador de campos. Por exemplo, se imos ler un ficheiro CSV (valores separados por comas), pode que queiramos usar a coma `,` ou o punto e coma `;` como delimitador para que cada elemento dunha liña sexa un obxecto a retornar. Ademais, desde PS 3.0 é posible empregar a opción `-TotalCount` para obter as *n* primeiras liñas, coma se fose o comando `head` de bash, namentres que a opción `-Tail` obtén as últimas liñas, coma o comando `tail` de bash. Vexamos algúns exemplos:

Obter as 4 primeiras liñas do arquivo `exemplo.txt`

```
Get-Content -Path .\exemplo.txt -TotalCount 4
```

Supoñendo que temos un ficheiro CSV con 3 ítems por liña, e queremos obter os elementos das dúas últimas liñas:

```
Get-Content -Path .\exemplo.csv -Delimiter "," -Tail 6
```

Se agora queremos modificar o contido dos arquivos, deberemos facer uso dos cmdlets `Clear-Content`, `Set-Content` e `Add-Content` para borrar, establecer, ou engadir contido respectivamente. O parámetro `-NoNewLine` do segundo e o terceiro serven para non introducir saltos de liña entre elementos ou ao final do arquivo. Estes cmdlets, ao igual que `Get-Content`, tamén teñen opcións para tratar con distintos tipos de codificacións de caracteres, se fora necesario, mais non imos profundizar sobre isto neste curso.

```
Clear-Content -Path <RUTA>
```

```
Set-Content -Path <RUTA> -Value <CONTIDO> [-NoNewLine]
```

```
Add-Content -Path <RUTA> -Value <CONTIDO> [-NoNewLine]
```

Por exemplo:

```
Clear-Content -Path .\exemplo.txt
```

```
Set-Content -Path .\exemplo.txt -Value Ola -NoNewLine
```

```
Add-Content -Path .\exemplo.txt -Value ", que tal?"
```

Unha vez que terminamos de operar co texto dos arquivos, pasamos ao cmdlet *Rename-Item*, que serve para renomear un ficheiro.

```
Rename-Item [-Path <RUTA>] [-NewName <NOME>] [-Force]
```

Por exemplo:

```
Rename-Item -Path "exemplo.txt" -NewName "exemplo1.txt"
```

Para copiar e mover ficheiros ou directorios, empregamos os cmdlets *Copy-Item* e *Move-Item* respectivamente:

```
Copy-Item [-Path <RUTA>] [-Destination <DESTINO>] [-Recurse] [-Force]
```

```
Move-Item [-Path <RUTA>] [-Destination <DESTINO>] [-Force]
```

Temos que facer notar que o cmdlet *Copy-Item* ten unha opción *-Recurse* para que os descendentes dun cartafol tamén se copien, mais o cmdlet *Move-Item* carece deste parámetro xa que é o comportamento que se lle presupón por defecto. Vexamos algúns exemplos:

```
Copy-Item -Path .\exemplo1.txt -Destination .\cartafol1
```

```
Copy-Item -Path .\cartafol1 -Destination .\cartafol2 -Recurse -Force
```

Ou se quixeramos mover en lugar de copiar:

```
Move-Item -Path cartafol1 -Destination cartafol2 -Force
```

Agora imos empregar o cmdlet *Get-ChildItem* para listar directorios. O parámetro *-Recurse* emprégase para listar tamén os subdirectorios do directorio obxectivo.

```
Get-ChildItem [-Path <RUTA>] [-Include <PATRÓN>] [-Exclude <PATRÓN>] [-Recurse]
```

Por exemplo, imos procurar no directorio actual os arquivos que inclúan a palabra exemplo e que non teñan extensión .png.

```
Get-ChildItem -Include *exemplo* -Exclude *.png
```

Agora, imos listar o cartafol "cartafol2", sen e con recursividade.

```
Get-ChildItem -Path cartafol2
```

```
Get-ChildItem -Path cartafol2 -Recurse
```

Para borrar un ficheiro ou directorio empregamos o cmdlet *Remove-Item*:

```
Remove-Item [-Path <RUTA>] [-Include <PATRÓN>] [-Exclude  
<PATRÓN>] [-Recurse]
```

Por exemplo:

```
Remove-Item -Path .\cartafol2\cartafol1\* -Include *.txt
```

```
Remove-Item -Path .\cartafol2 -Recurse
```

Para finalizar, imos ver un par de comandos para comprimir e descomprimir arquivos zip, que son *Compress-Archive* e *Expand-Archive* respectivamente. Estes dous comandos están dispoñibles unicamente desde PS 5. O comando *Compress-Archive* permite distintos niveis de compresión a través do parámetro *-CompressionLevel* (NoCompression, Fastest e Optimal). Ademais, se un arquivo comprimido contén certos arquivos e pretendemos introducir outros co mesmo nome, este cmdlet tamén permite que se actualicen os arquivos á súa versión máis recente a través do parámetro *-Update*.

```
Compress-Archive -Path <ARQUIVOS> -DestinationPath  
<ARQUIVO_COMPRIMIDO> [-CompressionLevel <NIVEL_COMPRESIÓN>] [-  
Force] [-Update]
```

```
Expand-Archive -Path <ARQUIVO_COMPRIMIDO> -DestinationPath  
<DESTINO> [-Force]
```

Por exemplo:

```
Compress-Archive -Path exemplo1.txt, exemplo2.txt -  
DestinationPath comprimido.zip -CompressionLevel Optimal
```

```
Compress-Archive -Path exemplo1.txt, exemplo2.txt -  
DestinationPath comprimido.zip -CompressionLevel Optimal -Update
```

```
Expand-Archive -Path .\comprimido.zip -DestinationPath C:\
```

Finalmente, salientaremos que en PowerShell 3 e 4 existe unha forma de comprimir e descomprimir arquivos a través dos comandos *New-ZipFile* e *Expand-ZipFile* [4], mais é necesario facer unha certa configuración para poder empregar estes cmdlets.

Referencias:

[1] Jonathan Hassel. "All about PowerShell providers and modules". En liña: <https://www.computerworld.com/article/3154158/all-about-powershell-providers-and-modules.html>. Accedido o 19-04-2019.

[2] m_power e brianstist en StackOverflow. "LiteralPath option for cmdlet". En liña: <https://stackoverflow.com/questions/28611307/literalpath-option-for-cmdlet>. Accedido o 19-04-2019.

[3] PowerShell Team. "Exclude, Include, Filter Parameters - How to make sense of these". En liña: <https://devblogs.microsoft.com/powershell/exclude-include-filter-parameters-how-to-make-sense-of-these/>. Accedido o 19-04-2019.

[4] SS64.com “New-Zipfile, Expand-ZipFile”. En liña: <https://ss64.com/ps/zip.html>. Accedido o 19-04-2019.

2.2 Cmdlets para tratar con procesos

Outro dos recursos máis importantes que debe xestionar un sistema operativo son os procesos. PowerShell inclúe unha serie de cmdlets para tratar con procesos, que veñen a substituír aos comandos do cmd *tasklist* e *taskkill*.

Para listar os procesos que están a correr no sistema, empregamos o cmdlet *Get-Process*:

```
Get-Process [-Name <NOMES_EXECUTABLES> | -Id <PIDs>] [-ComputerName <EQUIPOS>] [-FileVersionInfo] [-Module] [-IncludeUserName]
```

O parámetro *-Name* indica os nomes dos executables dos procesos a buscar. Trátase dunha lista separada por comas, e pode incluír caracteres comodín. En lugar deste parámetro tamén se pode empregar o parámetro *-Id*, que indica os PIDs do proceso que desexamos buscar. O parámetro *-FileVersionInfo* fai que se indique a versión do programa. Pola súa parte, o flag *-Module* indica que desexamos listar os módulos DLL cargados por este proceso. Finalmente, o flag *-IncludeUserName* fai que o resultado inclúa o nome do usuario que executa cada proceso. Este parámetro foi introducido en PS 4.0 e é incompatible con outros coma *-Module* e *-FileVersionInfo*.

Exemplos:

```
Get-Process
```

```
Get-Process -Name WordPad, explorer
```

```
Get-Process -Name "sql*" -FileVersionInfo
```

```
Get-Process -Id 2420 -Module
```

```
Get-Process -IncludeUserName
```

Para lanzar un novo proceso, dispoñemos do cmdlet *Start-Process*:

```
Start-Process -FilePath <RUTA_EXECUTABLE> [-ArgumentList <ARGUMENTOS>] [-Credential <PSCredential>] [-LoadUserProfile] [-NoNewWindow] [-UseNewEnvironment] [-Wait] [-WindowStyle {Normal | Hidden | Minimized | Maximized}] [-WorkingDirectory <String>]
```

O parámetro *-FilePath* indica a traxectoria ao executable en cuestión. Pola súa parte, o parámetro *-ArgumentList* son os argumentos que se lle pasaría ao executable se o invocásemos pola consola, separados por comas. O parámetro *-Credential* indica as credenciais do usuario que invoca o proceso, en caso de que queiramos executalo en nome doutro usuario. O flag *-LoadUserProfile* indica se se desexa cargar o perfil de usuario almacenado no rexistro de Winodws. O flag *-NoNewWindow* indica que non se desexa executar o proceso nunha nova ventá. Outro flag é *-UseNewEnvironment*, que indica que este proceso quere executarse cun novo conxunto de variables de entorno, en lugar de usar as do usuario e do

equipo. O flag `-Wait`, por outra banda, indica que queremos que a sesión de PowerShell se quede agardando ata que o proceso finalice. O parámetro `-WindowStyle`, indica cómo queremos que estea a ventá ao arrancar o proceso. Por último, o parámetro `-WorkingDirectory` especifica o directorio onde se estaría a executar o proceso. Exemplos:

```
Start-Process -FilePath cmd -Wait
```

```
Start-Process -FilePath msg -ArgumentList *,'Hola' -Credential  
"ucha.local\Administrador" -UseNewEnvironment
```

```
Start-Process -FilePath Wordpad -WindowStyle Maximized -  
WorkingDirectory "C:\Users\Administrador\Desktop"
```

O cmdlet *Stop-Process*, pola súa parte, serve para parar un ou varios procesos. Co parámetro `-Force` a parada faise forzosa.

```
Stop-Process [-Name <NOMES_EXECUTABLES> | -Id <PID>] [-Force]
```

Por exemplo:

```
Stop-Process -Name WordPad -Force
```

```
Stop-Process -Id 3342
```

O cmdlet *Wait-Process*, fai que a sesión de PowerShell quede agardando ata que un ou varios procesos rematen, coma se incluísemos a opción `-Wait` cando creáramos o proceso con *Start-Process*. O parámetro `-Timeout` fai que o bloqueo quede liberado se pasa un certo tempo:

```
Wait-Process -Name <NOMES_EXECUTABLES> | -Id <PIDs> [-Timeout  
<TEMPO_EN_SEGS>]
```

Por exemplo:

```
Wait-Process -Name Notepad
```

```
Wait-Process -Id 3384 -Timeout 30
```

2.3 Cmdlets para traballar con servizos

Aparte dos cmdlets para procesos, PowerShell tamén dispón doutro grupo para traballar con servizos, que ven substituíndo ao `sc` do `cmd`.

Primeiro, para obter os servizos do sistema dispoñemos do cmdlet *Get-Service*. Este cmdlet ten algúns parámetros de interese coma `-DependentServices`, que lista os servizos que dependen daqueles que buscamos, e `-RequiredServices`, que amosa os servizos requiridos por aqueles que buscamos.

```
Get-Service [-Name <NOMES> | -DisplayName <NOMES>] [-  
ComputerName <EQUIPOS>] [-DependentServices] [-RequiredServices]  
[-Exclude <PATRÓNS>] [-Include <PATRÓNS>]
```

Por exemplo:

```
Get-Service
```

```
Get-Service -Name TermService
```

```
Get-Service -Name TermService -RequiredServices
```

```
Get-Service -Name TermService -DependentServices
```

```
Get-Service -DisplayName "*Media Center*"
```

Para crear un novo servizo, usamos o cmdlet *New-Service*. Incluiremos parámetros para o nome (-Name) e o executable (-BinaryPathName), e opcionalmente para o nome visual (-DisplayName) e a descrición (-Description). Ademais, podemos indicar que o noso servizo depende doutros mediante o parámetro -DependsOn, e tamén podemos indicar o modo de arranque do noso servizo mediante o parámetro -StartupType. Unha vez creado o noso servizo, podemos cambiar a súa configuración mediante o cmdlet *Set-Service*, que é moi parecido.

```
New-Service -Name <NOME> -BinaryPathName <PATH_EXECUTABLE> [-Credential <CREDENCIAIS>] [-DependsOn <DEPENDENCIAS>] [-Description <DESCRICIÓN>] [-DisplayName <NOME>] [-StartupType {Boot | System | Automatic | Manual | Disabled}]
```

```
Set-Service -Name <NOME> [-ComputerName <String[]>] [-Description <DESCRICIÓN>] [-DisplayName <NOME>] [-StartupType {Boot | System | Automatic | Manual | Disabled}] [-Status {Running | Stopped | Paused}]
```

Por exemplo, imos crear un servizo “Proba” que executa o host de servizo de Windows, que depende do servizo NetLogon e que arranca de forma manual:

```
New-Service -Name "Proba" -BinaryPathName "C:\WINDOWS\System32\svchost.exe -k netsvcs" -DependsOn NetLogon -DisplayName "Servizo de proba" -StartupType Manual -Description "Este é un servizo de proba."
```

Agora cambiamos a descrición do noso servizo

```
Set-Service -Name "Proba" -Description "Este é o meu servizo de proba."
```

Para arrancar o noso servizo dispoñemos do cmdlet *Start-Service*. Para pausalo, *Suspend-Service*. Para reanudalo, *Resume-Service*, Para paralo, *Stop-Service*. Para reinicialo, *Restart-Service*. Os parámetros destes cmdlets son bastante similares, coa excepción de *Stop-Service* e *Restart-Service* que inclúen o parámetro -Force para forzar a parada do servizo, e que o cmdlet *Stop-Service* ten un flag -NoWait, que foi incluído en PS 5, e que serve para que retornemos inmediatamente á sesión de PowerShell en lugar de ter que agardar a que o servizo remate [1].

```
Start-Service -Name <NOMES> | -DisplayName <NOMES> [-Exclude <PATRÓNS>] [-Include <PATRÓNS>]
```

```
Suspend-Service -Name <NOMES> | -DisplayName <NOMES> [-Exclude <PATRÓNS>] [-Include <PATRÓNS>]
```

```
Resume-Service -Name <NOMES> | -DisplayName <NOMES> [-Exclude <PATRÓNS>] [-Include <PATRÓNS>]
```

```
Stop-Service -Name <NOMES> | -DisplayName <NOMES> [-Force] [-NoWait] [-Exclude <PATRÓNS>] [-Include <PATRÓNS>]
```

```
Restart-Service -Name <NOMES> | -DisplayName <NOMES> [-Force] [-Exclude <PATRÓNS>] [-Include <PATRÓNS>]
```

Probemos agora a xogar co servizo defragSvc, que inicialmente debería estar parado, xa que o servizo que creamos anteriormente pode tardar moito en iniciarse. Nótese que a maioría dos servizos que están correndo en Windows non soportan as operacións *Suspend-Service* e *Resume-Service*:

```
Start-Service -Name defragSvc
```

```
Suspend-Service -Name defragSvc
```

```
Resume-Service -Name defragSvc
```

```
Restart-Service -Name defragSvc
```

```
Stop-Service -Name defragSvc -NoWait
```

Finalmente, desde PS 6, dispoñemos dun cmdlet *Remove-Service* para quitar un servizo:

```
Remove-Service -Name <NOME>
```

Por exemplo:

```
Remove-Service -Name Proba
```

Mais en versións anteriores só temos dúas maneiras alternativas de eliminalo [2]. A primeira é usar o vello comando do cmd:

```
sc delete proba
```

A segunda é usar a API de Windows Media Instrumentation (WMI) para obter o servizo, e chamar ao método `delete()` do servizo retornado.

```
$service = Get-WmiObject -Class Win32_Service -Filter "Name='Proba'"
```

```
$service.delete()
```

Nota: no meu caso, o cmdlet *Get-Service* seguía a atopar o servizo ata que reinicie o sistema.

Referencias:

[1] Mike F. Robbins. "PowerShell Version 5 New Feature: NoWait Parameter added to Stop-Service". En liña: <https://mikefrobbins.com/2014/06/10/powershell-version-5-new-feature-nowait-parameter-added-to-stop-service/>. Accedido o 21-04-2019.

[2] P. Campbell, Adrian Russell, Thomas e ravikanth en stackoverflow. "Check if a Windows service exists and delete in PowerShell". En liña: <https://stackoverflow.com/questions/4967496/check-if-a-windows-service-exists-and-delete-in-powershell>. Accedido o 21-04-2019

2.4 Cmdlets para xestionar traballos en segundo plano

Ao igual que bash, PowerShell tamén ten unha serie de cmdlets para manexar os traballos en segundo plano. Para empezar, o cmdlet Get-Job serve para listar os traballos en segundo plano.

```
Get-Job [-Id <IDS> | -Name <NOMBRES> | -InstanceId <IDS> | -State {NotStarted | Running | Completed | Failed | Stopped | Blocked | Suspended | Disconnected | Suspending | Stopping | AtBreakpoint}] [-After <FECHA_HORA>] [-Before <FECHA_HORA>] [-ChildJobState {NotStarted | Running | Completed | Failed | Stopped | Blocked | Suspended | Disconnected | Suspending | Stopping | AtBreakpoint}] [-HasMoreData <BOOLEANO>] [-IncludeChildJob] [-Newest <NÚMERO>]
```

É posible buscar os traballos por id de traballo (-Id), por nome (-Name), por id de instancia (-InstanceId) ou polo estado (-State). Tamén é posible buscar polo comando que se executa durante o traballo (-Command), por datas e horas mediante os parámetros -After e -Before, polo estado do traballo fillo con -ChildJobState e por aqueles que teñen datos pendentes de entregar (-HasMoreData). Por último é posible buscar só os n últimos traballos con -Newest.

Por exemplo:

```
Get-Job
```

```
Get-Job -Id 1
```

```
Get-Job -Name Job1
```

```
Get-Job -Command Get-Process
```

```
Get-Job -State Running
```

Para lanzar un traballo en segundo plano, usamos o cmdlet *Start-Job*:

```
Start-Job [-ScriptBlock <SCRIPT> | -FilePath <RUTA>] [[-InitializationScript] <SCRIPT>] [-ArgumentList <ARGUMENTOS>] [-Authentication {Default | Basic | Negotiate | NegotiateWithImplicitCredential | Credssp | Digest | Kerberos}] [-Credential <CREDENCIAIS>] [-Name <NOME>] [-RunAs32]
```

O parámetro -ScriptBlock contén o código a executar en segundo plano. Alternativamente, podemos pasar a ruta a un ficheiro. Os argumentos pódense pasar en -ArgumentList. Se

necesitamos cargar funcións ou módulos de PS, ou calquera outra inicialización antes de lanzar o traballo, podemos empregar o parámetro `-InitializationScript` con este propósito. O parámetro `-Authentication` indica o mecanismo de autenticación de credenciais. Por último, o flag `-RunAs32` indica que o traballo se executa como un proceso de 32 bits, pero isto pode provocar problemas ao utilizar outras credenciais de autenticación en versións de 64 bits de Windows 7 e Windows 2008 R2.

Por exemplo, poderíamos lanzar un `Get-Process` en segundo plano, pero este acabará inmediatamente:

```
Start-Job -scriptblock {Get-Process} -Name "Busca Procesos"
```

Se queremos un traballo en segundo plano que non termine rapidamente para poder traballar con calma, podemos facer un bucle infinito:

```
Start-Job -scriptblock {while($true) {$i++}} -Name Infinito
```

Agora xa podemos buscalos mediante `Get-Job` e comparar os resultados.

```
Stop-Job [ -Filter <TÁBOA_HASH> | -Id <IDS> | -InstanceId <IDS> |  
| -Job <TRABALLOS> | -Name <NOMES> | -State {NotStarted |  
Running | Completed | Failed | Stopped | Blocked | Suspended |  
Disconnected | Suspending | Stopping | AtBreakpoint}]
```

Para deter un traballo, empregamos o cmdlet `Stop-Job`. Podemos detelos por ids, nomes, ids de instancias, estado etc. Por exemplo:

```
Stop-Job -id 1
```

```
Stop-Job -name Infinito
```

```
Stop-Job -state Running
```

Cando o resultado dun traballo está listo, podemos querer obtelo mediante o cmdlet `Receive-Job`, xa sexa buscando por Id, por nome, por id de instancia ou pasando directamente o traballo polo pipeline.

```
Receive-Job -Id <IDS> | -InstanceId <IDS> | -Job <TRABALLOS> | -  
Name <NOMES> [[-ComputerName <String[]>] [-AutoRemoveJob] [-  
Force] [-Keep] [-NoRecurse] [-Wait] [-WriteEvents] [-  
WriteJobInResults]
```

O resultado elimínase da cola ao recollelo, a menos que empreguemos o parámetro `-Keep`. O flag `-AutoRemoveJob` elimina o traballo de paso. O parámetro `-Force` indica que se seguirá agardando por resultados incluso se os traballos están en estado suspendido ou desconectado. O flag `-NoRecurse` indica que queremos o resultado dun certo traballo, pero non dos seus descendentes. O flag `-Wait` indica que a sesión de PS queda suspendida ata que os resultados estean dispoñibles. O flag `-WriteEvents` indica que se o estado dun traballo cambia mentras se agarda por el, este cambio será notificado. Finalmente, o flag `-WriteJobInResults` indica que o propio traballo será devolto como parte dos resultados. Algúns parámetros, coma `-Keep` e `-Wait` son incompatibles entre si. Outros, só se poden empregar se se emprega certo

parámetro, como `-AutoRemoveJob` e `-WriteJobInResults`, que só se poden empregar con `-Wait`. Vexamos uns exemplos:

```
Receive-Job -id 1 -Keep
```

```
Receive-Job -id 1 -Wait -AutoRemoveJob -WriteJobInResults
```

```
Receive-Job -name Infinito
```

Se queremos bloquear a sesión de PowerShell ata que un ou varios traballos terminen, só temos que empregar o cmdlet *Wait-Job*, que pode empregar o id dos traballos, o seu nome, o id de instancia, o estado, un filtro, ou o propio traballo pasado por pipeline:

```
Wait-Job -Filter <FILTRO> | -Id <IDS> | -InstanceId <IDS> | -Job  
<TRABALLOS> | -Name <NOMES> | -State {NotStarted | Running |  
Completed | Failed | Stopped | Blocked | Suspended |  
Disconnected | Suspending | Stopping | AtBreakpoint} [-Any] [-  
Force] [-Timeout <TEMPO_SEG>]
```

O flag `-Any` indica que o bloqueo se liberará en canto un dos traballos remate. O parámetro `-Timeout` indica que o bloqueo pasado un tempo establecido. O flag `-Force` fai que quede agardando por traballos suspendidos ou desconectados. Vexamos uns exemplos:

```
Wait-Job -id 1
```

```
Wait-Job -name Infinito -Timeout 20
```

```
Wait-Job -state Running -Any
```

Hai que ter en conta que esperar sen timeout por un traballo infinito vai a bloquear a devandita consola.

Por último, para borrar un traballo, temos o cmdlet *Remove-Job*, que pode empregar o id dos traballos, o seu nome, o id de instancia, o estado, un filtro, os comandos ou o propio traballo pasado por pipeline:

```
Remove-Job -Command <COMANDOS> -Filter <FILTRO> | -Id <IDS> | -  
InstanceId <IDS> | -Job <TRABALLOS> | -Name <NOMES> | -State  
{NotStarted | Running | Completed | Failed | Stopped | Blocked |  
Suspended | Disconnected | Suspending | Stopping | AtBreakpoint}  
[-Force]
```

Por defecto, este cmdlet non borra os traballos cuxo estado sexa `Running`, pero isto pódese cambiar mediante a opción `-Force` ou borrando explicitamente por estado. Vexamos uns exemplos:

```
Remove-Job -id 1
```

```
Remove-Job -name Infinito -Force
```

```
Remove-Job -state Running
```

2.5 Cmdlets para tratar con alias

Como xa se dixo anteriormente, PowerShell incorpora un mecanismo de alias para facilitar a transición dos usuarios de CMD e bash. Un alias é unha cadea que substitúe a outra. Pode substituír a un comando, un script, unha función ou un arquivo executable. Mais a diferenza de bash, non pode substituír a un comando con parámetros. Por exemplo, non podería substituír a:

```
Get-Command -name get-*
```

Internamente, un alias é un obxecto da clase *System.Management.Automation.AliasInfo* e ten un ámbito local, global ou de script. Para ver os aliases instalados no sistema, basta con invocar o cmdlet *Get-Alias*. O parámetro *-Scope* restrinxe os ámbitos aos que se aplica o alias, xa que algúns son globais, outros son locais á sesión e outros só pertencen ao script que se estea executando.

```
Get-Alias [-Name <NOME> | -Definition <DEFINICIÓN>] [-Exclude <FILTRO>] [-Scope <ALCANCE>]
```

Por exemplo:

```
Get-Alias
```

```
Get-Alias -Name ls
```

```
Get-Alias -Definition Get-ChildItem -Scope Global
```

Para establecer un alias, empregamos os cmdlets *New-Alias* e *Set-Alias*. Estes comandos son practicamente iguais, coa diferenza de que *New-Alias* falla se o alias xa existía.

```
New-Alias -Name <NOME> -Value <VALOR> [-Description <DESCRICIÓN>] [-Force] [-Option {None | ReadOnly | Constant | Private | AllScope | Unspecified}] [-Scope <String>]
```

```
Set-Alias -Name <NOME> -Value <VALOR> [-Description <DESCRICIÓN>] [-Force] [-Option {None | ReadOnly | Constant | Private | AllScope | Unspecified}] [-Scope <ALCANCE>]
```

Por exemplo:

```
New-Alias -Name listar -Value Get-ChildItem -Scope local
```

```
Set-Alias -Name listar -Value Get-ChildItem -Description "Comando para listar" -Scope local
```

Para eliminar un alias, desde PowerShell 6 dispoñemos dun cmdlet *Remove-Alias*. Porén, en versións anteriores de PowerShell, temos que empregar o cmdlet *Remove-Item* e no nome de ítem a eliminar, temos que poñer o prefixo "alias:". Por exemplo:

```
Remove-Item alias:\listar
```

Finalmente, se queremos gardar os nosos alias e volvelos a empregar noutras sesións, podemos exportalos a ficheiro mediante o cmdlet *Export-Alias*, e volvelos a importar mediante o cmdlet *Import-Alias*:

```
Export-Alias -Path <RUTA> [-Name <NOMES_ALIASES>] [-Append] [-As {Csv | Script}] [-Description <DESCRICIÓN>] [-Force] [-Scope <ALCANCE>]
```

```
Import-Alias -Path <RUTA> [-Force] [-Scope <ALCANCE>]
```

O parámetro *-As* permite escoller o formato no que serán exportados os alias. Por exemplo:

```
Export-Alias -Path .\meus_alias.csv -Name listar -As CSV -Scope local
```

```
Import-Alias -Path .\meus_alias.csv -Name listar -Scope local
```

Este último cmdlet dará error se o alias aínda existe.

2.6 Listado de cmdlets de Windows PowerShell

A continuación ofrécense un compendio dos comandos de Windows PowerShell. Malia que a maioría de listados ofrécense por orde alfabético sobre o comando completo, nós ofrecerémolo ordenando alfabeticamente pola segunda palabra do comando, é dicir, a que vai despois do guión. Con isto podemos encontrar os comandos que pertencen a unha certa temática con maior facilidade.

A

Comando	Alias	Función
Get-Acl		Obtén os permisos dun arquivo ou clave de rexistro
Set-Acl		Establece os permisos dun arquivo ou clave de rexistro
Export-Alias	epal	Exporta unha lista de alias a un arquivo
Get-Alias	gal	Retorna os alias de Cmdlets
Import-Alias	ipal	Importa unha lista de alias dun arquivo
New-Alias	nal	Crea un novo alias
Set-Alias	sal	Crea ou cambia un alias
Compress-Archive		Crea un novo arquivo comprimido/zip [PS 5+]
Expand-Archive		Extrae arquivos dun ficheiro comprimido/zip [PS 5+]
Get-AuthenticodeSignature		Obtén o obxecto de firma asociado a un arquivo
Set-AuthenticodeSignature		Pon unha firma nun ficheiro de script .ps1, etc.

C

Comando	Alias	Función
Get-ChildItem	dir/ls/gci	Obtén os obxectos fillos dun directorio ou dunha clave de rexistro
Export-Clixml		Produce unha representación clixml de obxectos de PS
Import-Clixml		Importa un arquivo clixml e refai o proxecto

		de PS
Get-Command	Gcm	Obtén información básica dun comando
Invoke-Command	Icm	Invoca un comando
Measure-Command		Mide o tempo de execución dun comando
Trace-Command		Devolve a traza dunha expresión ou comando
Add-Computer		Engade un ordenador ao dominio
Checkpoint-Computer		Crea un punto de restauración (XP)
Remove-Computer		Quita unha máquina dun dominio ou dun grupo de traballo
Restart-Computer		Reinicia o sistema operativo dunha máquina
Restore-Computer		Restaura un ordenador a un estado previo
Stop-Computer		Para (apaga) un ordenador
Reset-ComputerMachinePassword		Restablece o contrasinal dunha máquina
Test-ComputerSecureChannel		Repara a canle segura a un dominio
Enable-ComputerRestore		Habilita la restauración dunha unidade
Disable-ComputerRestore		Deshabilita a restauración dunha unidade
Get-ComputerRestorePoint		Obtén los puntos de restauración
Export-Console		Exporta a config. dunha consola a ficheiro
Add-Content	Ac	Engade ao contido dun ítem
Get-Content	cat/type/gc	Obtén contido dun ítem
Set-Content	Sc	Establece o contido dun ítem
Clear-Content	Clc	Borra contido dun ítem/arquivo
Test-Connection		Fai ping a unha ou máis máquinas
Export-Counter		Exporta datos do Contador de Rendemento a ficheiros de log
Get-Counter		Obtén datos do Contador de Rendemento
Import-Counter		Importa ficheiros de log de Contador de Rendemento
Get-Credential		Obtén unha credencial de seguridade (usuario/contrasinal)
ConvertFrom-CSV		Crea obxectos de texto en formato CSV
ConvertTo-CSV		Converte obxectos .NET en texto CSV
Export-CSV	Epcsv	Exporta un obxecto de PowerShell a una arquivo CSV
Import-CSV	lpcsv	Toma obxectos dunha lista en CSV e mándaos ao pipeline
Get-Culture		Obtén información rexional
Format-Custom		Formatea a saída cun formato especial

D

Comando	Alias	Función
Get-Date		Obtén a data e hora actual
Set-Date		Establece a data e hora do sistema
Write-Debug		Escribe unha mensaxe de depuración no host/pantalla
Out-Default		Manda a saída ao lugar por defecto

Get-DnsClientCache		Obtén o contido da caché do cliente local DNS
Clear-DnsClientCache		Limpa o contido da caché do cliente DNS
Get-DscConfiguration		Obtén a configuración actual dun nodo [PS 4+]
Get-DscLocalConfigurationManager		Obtén as opcións do Xestor de configuración local para un nodo [PS 4+]
Get-DscResource		Obtén os recursos de configuración desexados dunha máquina
New-DscChecksum		Crea arquivos checksum para docs/recursos DSC
Start-DscConfiguration		Aplica config. de estado desexada a nodos [PS 4+]

E

Comando	Alias	Función
Register-EngineEvent		Subscríbese a eventos de PowerShell
Write-Error		Escribe un obxecto no pipeline de error
Get-Event		Obtén eventos da cola de eventos de PowerShell
New-Event		Crea un novo evento
Remove-Event		Borra un evento da cola de eventos
Unregister-Event		Cancela unha subscrición a un evento
Wait-Event		Espera ata que un certo evento ocorre
Clear-EventLog		Borra todas as entradas dun log de eventos
Get-EventLog		Obtén datos do log de eventos (2003)
Limit-EventLog		Limita o tamaño do log de eventos
New-EventLog		Crea un novo log de eventos e unha nova fonte de eventos
Remove-EventLog		Borra un evento da cola de eventos
Show-EventLog		Amosa un log de eventos
Write-EventLog		Escribe un evento nun log de eventos
Get-EventSubscriber		Obtén subscritores a eventos
Get-ExecutionPolicy		Obtén a política de execución do shell
Set-ExecutionPolicy		Establece a política de execución do shell
Invoke-Expression	iex	Executa una expresión de PowerShell

F

Comando	Alias	Función
Out-File		Manda a saída a un ficheiro. Como «>» pero con parámetros.
Unblock-File		Desbloquea ficheiros descargados de Internet [PS 3+]
Export-FormatData		Garda datos de formateado da sesión actual
Get-FormatData		Obtén os datos de formateado da sesión actual
Update-FormatData		Actualiza ficheiros de formateado de datos

G

Comando	Alias	Función
---------	-------	---------

Backup-GPO		Fai backup de obxectos de política de grupo (GPOs)
Import-GPO		Importa configuracións de políticas de grupo nunha certa GPO desde unha copia de seguridade
Restore-GPO		Restaura una GPO dunha copia de seguridade
Out-GridView	Ogv	Manda a saída a una táboa interactiva

H

Comando	Alias	Función
Get-Help	Help	Abre o ficheiro de axuda
Update-Help		Descarga e instala arquivos de axuda
Add-History		Engade entradas ao historial da sesión
Clear-History	Clhy	Borra entradas do historial da sesión
Get-History	history/h/ghy	Obtén un listado do historial de sesión
Invoke-History	r/ihy	Invoca un Cmdlet previamente executado
Clear-Host	clear/cls	Limpa a pantalla
Get-Host		Obtén información do host (versión de PS y rexión)
Out-Host	Oh	Manda a saída ao host
Read-Host		Lee unha liña metida na entrada da consola
Write-Host		Escrebe a saída no host/pantalla
Get-HotFix		Obtén os parches instalados
ConvertTo-HTML		Converte a entrada nunha táboa HTML

I

Comando	Alias	Función
Clear-Item	Cl	Quita o contido dunha variable ou un alias
Copy-Item	copy/cp/ci	Copia un ítem dunha localización ao espazo de nomes
Get-Item	Gi	Obtén un obxecto ficheiro ou valor de rexistro
Invoke-Item	li	Invoca un executable ou abre un arquivo (START)
Move-Item	mv/move/mi	Move un ítem dun lugar a outro
New-Item	md/mkdir/ni	Crea un novo ítem nun espazo de nomes
Remove-Item	del/erase/ rd/rm/ri/ rmdir	Borra un ítem
Rename-Item	ren/rni	Cambia o nome dun ítem existente
Set-Item	si	Cambia o valor dun ítem
Clear-ItemProperty	clp	Quita o valor dunha propiedade
Copy-ItemProperty	cpp	Copia unha propiedade e o seu valor
Get-ItemProperty	gp	Obtén as propiedades dun obxecto
Move-ItemProperty	mp	Move unha propiedade dun lugar a outro
New-ItemProperty		Establece unha nova propiedade
Remove-ItemProperty	rp	Borra unha propiedade e o seu valor
Rename-ItemProperty	rnp	Renomea a propiedade dun ítem
Set-ItemProperty	sp	Establece unha propiedade a un determinado valor

J

Comando	Alias	Función
Get-Job	gjb	Obtén as tarefas que se están a executar en segundo plano
Receive-Job	rcjb	Obtén os resultados de tarefas executándose en segundo plano
Remove-Job	rjb	Borra unha tarefa que se está executando en segundo plano
Start-Job	sajb	Comeza unha tarefa en segundo plano
Stop-Job	spjb	Para unha tarefa que se está a executar en segundo plano
Wait-Job	wjb	Agarda por unha tarefa que se está a executar en segundo plano

L

Comando	Alias	Función
Format-List	fl	Formatea a saída como unha lista, cada propiedade nunha liña
Update-List		Engade e quita elementos dunha colección
Get-Location	pwd/gl	Obtén e amosa o directorio de traballo actual
Pop-Location	popd	Establece o directorio de traballo tomándoo dunha pila
Push-Location	pushd	Pon o directorio de traballo na cima dunha pila
Set-Location	cd/chdir/sl	Establece o directorio de traballo actual

M

Comando	Alias	Función
Send-MailMessage		Manda unha mensaxe por email
Add-Member		Engade un membro a unha instancia dun obxecto PS
Get-Member	gm	Enumera as propiedades dun obxecto
Get-Module	gmo	Obtén os módulos que se importaron na sesión
Import-Module	ipmo	Engade un módulo á sesión actual
New-Module	nmo	Crea un novo módulo dinámico (só en memoria)
Remove-Module	rmo	Quita un módulo da sesión actual
Export-ModuleMember		Exporta membros de módulos específicos
Set-MpPreference		Configura as preferencias de Windows Defender

N

Comando	Alias	Función
Out-Null		Manda a saída a null (ningures)

O

Comando	Alias	Función
Compare-Object	diff/compare	Compara propiedades de obxectos
ForEach-Object	%	Fai unha operación sobre cada elemento dunha colección de obxectos
Group-Object	group	Agrupar obxectos que conteñen os mesmos valores
Measure-Object		Mide as propiedades un obxecto
New-Object		Crea un novo obxecto .NET
Select-Object	select	Selecciona propiedades de obxectos

Sort-Object	sort	Ordena propiedades de obxectos por valor
Tee-Object	tee	Manda obxectos de entrada a dous lugares
Where-Object	where/?	Filtra obxectos pasados polo pipeline
Register-ObjectEvent		Subscribe a eventos de .NET
Write-Output	write/echo	Escribe un obxecto no pipeline

P

Comando	Alias	Función
Convert-Path	cvpa	Converte unha ruta de PS nunha ruta de provedor
Join-Path		Combina unha ruta e unha ruta filla
Resolve-Path	rvpa	Resolve os comodíns nunha ruta
Split-Path		Devolve parte dunha ruta
Test-Path		Devolve «true» se a ruta existe, «false» en caso contrario
Pause		Pausa e amosa a mensaxe «Prema intro para continuar»
Get-Pfxcertificate		Obtén información acerca do certificado pfx
Powershell		Lanza unha sesión de PowerShell ou executa un script
Get-Process	ps/gps	Obtén unha lista dos procesos nesa máquina
Debug-Process		Inclúe un depurador nun proceso en execución
Start-Process	start/saps	Comeza un ou máis procesos
Stop-Process	kill/spps	Para un proceso en execución
Wait-Process		Espera a que un proceso pare para continuar a sesión
Out-Printer	lp	Manda a saída á impresora
Write-Progress		Amosa unha barra de progreso
Enable-PSBreakpoint	ebp	Habilita un breakpoint na consola actual
Disable-PSBreakpoint	dbp	Deshabilita un breakpoint na consola actual
Get-PSBreakpoint	gbp	Obtén os breakpoints que se estableceran
Set-PSBreakpoint	sbp	Pon un breakpoint nuna liña, comando ou variable
Remove-PSBreakpoint	rbp	Borra breakpoints da consola actual
Set-PSDebug		Activa ou desactiva propiedades da depuración de scripts
Get-PSDrive	gdr	Obtén información do dispositivo (DriveInfo)
New-PSDrive	mount/ndr	Crea un dispositivo de rede mapeado
Remove-PSDrive	rdr	Quita un provedor/dispositivo do seu lugar actual
Get-PSProvider		Obtén información do provedor especificado
Disable-PSRemoting		Quita a configuración de sesión remota no ordenador
Enable-PSRemoting		Activa a sesión remota. Lanza comandos remotos
Enter-PSSession	etsn	Inicia unha sesión interactiva, local ou remota
Exit-PSSession	exit/exsn	Sae dunha sesión interactiva, local o remota
Export-PSSession	epsn	Importa comandos e gárdalos nun módulo de PS
Get-PSSession	gsn	Obtén as sesións de PS na sesión actual
Import-PSSession	ipsn	Importa comandos doutra sesión
New-PSSession	nsn	Crea unha conexión persistente para unha sesión
Remove-PSSession	rsn	Pecha sesións de PowerShell

Disable-PSSessionConfiguration		Deshabilita a configuración de sesións no ordenador local
Enable-PSSessionConfiguration		Habilita a configuración de sesións no ordenador local
Get-PSSessionConfiguration		Obtén a configuración de sesións no ordenador local
Register-PSSessionConfiguration		Crea e rexistra unha nova configuración de sesión PS
Set-PSSessionConfiguration		Cambia as propiedades da configuración dunha sesión rexistrada
Unregister-PSSessionConfiguration		Borra as sesións de PS rexistradas
New-PSSessionOption		Crea un obxecto con opcións avanzadas para unha sesión PS
Add-PSSnapIn	asnp	Engade snap-ins á consola
Get-PSSnapIn	gsnp	Lista os snap-ins nesta máquina
Remove-PSSnapIn	rsnp	Quita snap-ins nesta máquina

R

Comando	Alias	Función
Get-Random		Obtén un número aleatorio
Run / Call	&	Executa un comando (operador de chamada)

S

Comando	Alias	Función
ConvertFrom-SecureString		Converte cadeas seguras en cadeas cifradas estándar
ConvertTo-SecureString		Converte cadeas cifradas estándar en cadeas seguras
Out-String		Manda obxectos ao host como strings
Get-Service	gsv	Obtén unha lista de servizos
New-Service		Crea un novo servizo
Restart-Service		Para e reinicia un servizo
Resume-Service		Continúa un servizo suspendido
Set-Service		Cambia o modo de inicio ou propiedades dun servizo
Start-Service	sasv	Inicia un servizo parado
Stop-Service	spsv	Para un servizo en marcha
Suspend-Service		Suspende un servizo en marcha
Set-StrictMode		Forza as regras de codificación en expresións e scripts
Start-Sleep	sleep	Suspende a actividade do shell, script ou espazo de execución
ConvertFrom-StringData		Converte unha here-string (@"...") nunha táboa hash
Select-String		Busca patróns en cadeas ou arquivos

T

Comando	Alias	Función
Format-Table	ft	Formatea a saída como unha táboa
New-Timespan		Crea un obxecto timespan
Get-TraceSource		Obtén os compoñentes dos que se poden facer trazas
Set-TraceSource		Fai unha traza dun compoñente de PowerShell
Complete-Transaction		Fai commit a unha transacción
Get-Transaction		Obtén información sobre a transacción activa
Start-Transaction		Comeza unha nova transacción
Undo-Transaction		Desfai unha transacción (rollback)
Use-Transaction		Engade un comando ou expresión a unha transacción
Start-Transcript		Comeza unha transcrición dunha sesión de shell
Stop-Transcript		Para o proceso de transcrición
Add-Type		Engade un tipo .NET a unha sesión de shell
Update-TypeData		Actualiza a configuración de tipos extendidos

U

Comando	Alias	Función
Get-UICulture		Obtén a información rexional da UI
Get-Unique	gu	Obtén os elementos únicos dunha lista ordenada

V

Comando	Alias	Función
Clear-Variable	clv	Quita o valor dunha variable
Get-Variable	gv	Obtén unha variable PowerShell
New-Variable	nv	Crea unha nova variable
Remove-Variable	rv	Elimina unha variable e o seu valor
Set-Variable	set/sv	Establece unha variable e o seu valor
Write-Verbose		Escribe unha cadea no display verboso do host

W

Comando	Alias	Función
Checkpoint-WebApplicationMonitoring		Crea un checkpoint para unha aplicación web IIS
Get-WebApplicationMonitoringStatus		Obtén o estado de monitorización de aplicacións web [PS 4+]
New-WebServiceProxy		Crea un obxecto proxy dun servizo web
Invoke-WebRequest		Obtén o contido dunha páxina web
Add-WindowsFeature		Instala roles, servizos de rol e características
Get-WindowsFeature		Lista roles, servizos de rol e características
Remove-WindowsFeature		Desinstala roles, servizos de rol e características
Write-Warning		Escribe un aviso (en vídeo reverso) na pantalla
Format-Wide	fw	Formatea a saída como una táboa pero con

		só unha propiedade por fila
Get-WinEvent		Obtén datos do log de eventos (Vista+)
Register-WmiEvent		Subscribe a un evento WMI
Set-WmiInstance		Crea ou actualiza unha instancia dunha clase WMI
Invoke-WmiMethod	iwmi	Chama a métodos WMI
Get-WmiObject	gwmi	Obtén información sobre unha clase WMI
Remove-WmiObject	rwmi	Borra unha instancia dunha clase WMI
Connect-WSMan		Conecta co servizo WinRM dunha máquina remota
Disconnect-WSMan		Desconecta do servizo WinRM dunha máquina remota
Test-WSMan		Mira se unha máquina está habilitada para comandos remotos
Invoke-WSManAction		Invoca unha acción sobre un obxecto específico
Disable-WsManCredSSP		Deshabilita autenticación SSP
Enable-WsManCredSSP		Habilita autenticación SSP
Get-WsManCredSSP		Obtén a configuración de credenciais SSP
Get-WSManInstance		Amosa información de administración (XML ou valor)
New-WSManInstance		Crea unha nova instancia dun obxecto de administración
Remove-WSManInstance		Elimina unha instancia dun obxecto de administración
Set-WSManInstance		Modifica a información de administración dun recurso
Set-WSManQuickConfig		Configura a máquina local para administración remota
New-WSManSessionOption		Crea opcións para os comandos WSMan

X

Comando	Alias	Función
ConvertTo-XML		Convierte la entrada en texto XML
Select-XML		Busca texto en una cadena o documento XML